Programación 4 - 2026

Laboratorio 0 :: Conceptos Básicos en C++

Consideraciones generales:

- La entrega podrá realizarse hasta el **22 de marzo de 2026 a las 23:59hrs.**
- El código fuente y el archivo Makefile [1] deberán ser entregados mediante el EVA del curso [2] dentro de un archivo con nombre `<número de grupo>_lab0.zip` (o `tar.gz`).
- Más allá de que se sugiere el uso de un entorno de desarrollo integrado (IDE, por sus siglas en inglés) para contar con un ambiente de desarrollo adecuado, el archivo Makefile entregado debe ser independiente de cualquier IDE, permitiendo la compilación aislada de la solución.
- El código deberá poder compilarse y ejecutarse sin errores en las máquinas Linux de la Facultad de Ingeniería **con la flag del compilador -std=c++98[1] [3] (no estando permitido el uso de auto, range-based loops, etc. en los archivos definidos)**.
- Las entregas que no cumplan estos requerimientos no serán consideradas. El hecho de no realizar una entrega implica la insuficiencia del laboratorio completo.

Objetivo

El objetivo del Laboratorio 0 es reforzar conceptos básicos de orientación a objetos a través de su implementación en el lenguaje C++ [4], así como practicar diversas construcciones del lenguaje y del entorno de programación en Linux [5] que serán de utilidad en etapas posteriores del laboratorio. Se espera que se consulte el material disponible en el EVA del curso (ver referencias al final de este documento), complementado con consultas a otros medios (ej.: Internet) con espíritu crítico y corroborando que las fuentes consultadas sean confiables.

Problema

El diagrama de clases UML de la siguiente página representa el diseño de un sistema para la gestión de experiencias turísticas y sus clientes. En este sistema, se modelan distintos tipos de actividades recreativas y los turistas que las reservan, con el propósito de organizar y mantener un registro estructurado y centralizado de las ofertas y sus participantes.

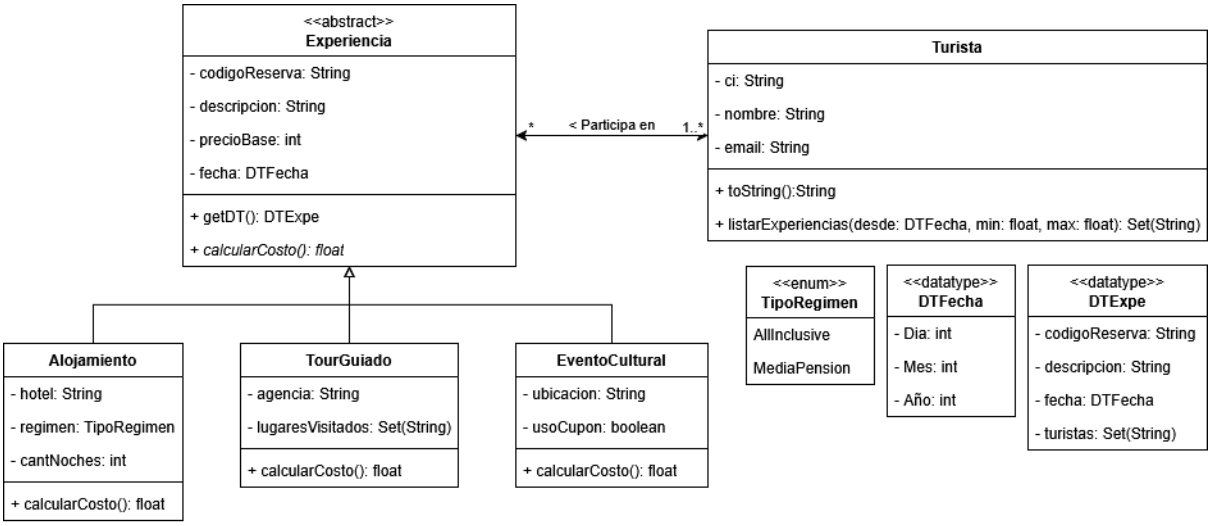
Para representar las experiencias, el sistema define una jerarquía de clases en la que la clase abstracta `Experiencia` funciona como la superclase de tres tipos específicos: `Alojamiento`, `TourGuiado` y `EventoCultural`. Cada experiencia cuenta con un identificador string único que funciona como código de reserva. Además, se almacena el nombre descriptivo de la actividad y la fecha en la que se llevará a cabo y el precio base.

Dependiendo del tipo de experiencia, se incluyen atributos adicionales dentro de sus respectivas clases: en un `Alojamiento`, se registra el nombre del hotel, un enumerado del régimen (All Inclusive, Media Pensión) y la cantidad de noches, mientras que en el caso de un `TourGuiado`, se

[1] Un ejemplo de llamado al compilador podría ser g++ -std=c++98 -o mi_programa main.cpp

almacena el nombre de la agencia responsable junto a una colección de strings que representan el itinerario de puntos a visitar. Por otro lado, para un `EventoCultural`, se registra la ubicación física y un booleano que indica si se utilizó un cupón de descuento.

Los turistas que participan en estas experiencias están representados por la clase `Turista`, que incluye un identificador único cédula de identidad, su nombre y su email.



El sistema modela la relación entre `Turista` y `Experiencia` mediante una asociación denominada *"participa en"*, que permite representar qué turistas participan en las experiencias. Asimismo, en la clase `Experiencia` se registran sus participantes asociados. En ambos casos se utiliza pseudo-atributos para manejar la colección de objetos asociados.

A nivel de comportamiento, se definen las siguientes operaciones:

- La operación `getDT()` en la clase `Experiencia` devuelve un datavalue de tipo `DTExpe`, que incluye el código de reserva, la descripción y la fecha del objeto, así como un listado de los nombres de los turistas relacionados.

- La operación `calcularCosto()` se encuentra en las clases `Experiencia`, `Alojamiento`, `TourGuiado` y `EventoCultural`. Se trata de una operación polimórfica que devuelve el costo total de la experiencia. En el caso de `Alojamiento` la operación da como resultado el precio base multiplicado por la cantidad de noches y se le agrega 10 unidades por cada noche si el régimen es all inclusive. Para los `TourGuiado`, la operación calcula el precio base más 2 unidades multiplicado por la cantidad de lugares a visitar multiplicado por la cantidad de turistas participantes. En el caso de `EventoCultural`, la operación retorna el precio base multiplicado por la cantidad de turistas participantes y se resta 5 unidades por cada turista si se utiliza cupón de descuento. Esta operación es abstracta en `Experiencia` y sus métodos están definidos en las clases `Alojamiento`, `TourGuiado` y `EventoCultural`.

- La operación `listarExperiencias(desde: DTFecha, min: float, max: float)` de la clase `Turista` obtiene el código de reserva de todas sus experiencias cuya fecha sea posterior al parámetro `desde` y costo sea mayor o igual que `min` y menor igual

que `max`.

- La operación `toString()` en la clase `Turista` devuelve un string de la forma: `ci->nombre/email`.

Se pide:

1. Implementar todas las clases (incluyendo sus atributos, pseudo-atributos, constructores, destructores, getters y setters que sean necesarios) y datatypes que aparecen en el diagrama.

2. Sobrecargar el operador de inserción de flujo (<<) en un objeto `std::ostream`.
   Esta sobrecarga debe permitir imprimir todos los datos del datatype `DTExpe` siguiendo el siguiente formato:
   `codigoReserva->descripcion(fecha)/turista1,turista2,.,turista N`

3. Implementar la operación `calcularCosto` en las clases `Experiencia`, `Alojamiento`, `TourGuiado` y `EventoCultural`.

4. Implementar la operación `toString` en la clase `Turista`.

5. Implementar la operación `listarExperiencias` en la clase `Turista`.

6. Implementar un método `main` que defina un conjunto de `Experiencia` y un conjunto de `Turista` y que ejecute la siguiente secuencia de pasos, donde se especifica cuáles deben imprimirse en la consola.

   a. Crear los siguientes objetos de la clase `Alojamiento` (con el constructor por parámetros):

codigoReserva	ALX5489
descripcion	Hotel moderno
precioBase	30
fecha	18/05/2020
hotel	Hotel Lindorf
regimen	AllInclusive
cantNoches	5

codigoReserva	ALJ4789
descripcion	Todas las habitaciones con vista al mar
precioBase	100
fecha	10/02/2025
hotel	Hotel SeaView
regimen	MediaPension
cantNoches	15

b. Crear los siguientes objetos de la clase `TourGuiado` (con el constructor por parámetros):

codigoReserva	TGO4657
descripcion	Plazas de Montevideo
precioBase	10
fecha	29/08/2024
agencia	Paseos SA
lugaresVisitados	Plaza Independencia Plaza Matriz

codigoReserva	TGR3257
descripcion	Puntos emblematicos
precioBase	5
fecha	29/08/2024
agencia	Recorre
lugaresVisitados	Puerta de la Ciudadela Mausoleo Cabildo Palacio Salvo

c. Crear los siguientes objetos de la clase `EventoCultural` (con el constructor por parámetros):

codigoReserva	ECP1346
descripcion	Danza en el Solis
precioBase	10
fecha	29/10/2025
ubicacion	Teatro Solis
usoCupon	true

d. Imprimir en consola utilizando la inserción de flujo el resultado de ejecutar la operación `getDT()` para cada uno de los objetos `Experiencia` creados.

e. Crear los siguientes objetos de la clase `Turista` (con el constructor por parámetros):

ci	4.951.278-9
nombre	Vanesa Castro
email	vcastro.uy@servidor.net

ci	1.535.442-0
nombre	Karen Santos
email	karen.s89@internet.uy

f. Imprimir en consola el resultado de ejecutar la operación `toString` para cada uno de los objetos `Turista` creados.

g. Registrar las siguientes relaciones entre turistas y experiencias (creando links de la relación en ambas direcciones).

Turista	Experiencia
4.951.278-9 (Vanesa Castro)	ALX5489 (Hotel moderno)
4.951.278-9 (Vanesa Castro)	ALJ4789 (Todas las habitaciones con vista al mar)
4.951.278-9 (Vanesa Castro)	TGR3257 (Puntos emblematicos)
4.951.278-9 (Vanesa Castro)	ECP1346 (Danza en el Solis)
1.535.442-0 (Karen Santos)	TGO4657 (Plazas de Montevideo)
1.535.442-0 (Karen Santos)	TGR3257 (Puntos emblematicos)

h. Invocar la operación `listarExperiencias(10/12/2023, 0, 1000)` para la turista 4.951.278-9 (Vanesa Castro) e imprimir el resultado en consola (un string por línea).

i. Ejecutar la eliminación del objeto TGR3257(Puntos emblematicos) de la clase `Experiencia`.

j. Invocar la operación `listarExperiencias(10/10/2020, 0, 1000)` para la turista 1.535.442-0 (Karen Santos) e imprimir el resultado en consola (un string por línea).

k. Imprimir en consola utilizando la inserción de flujo el resultado de ejecutar la operación `getDT()` para cada uno de los objetos `Experiencias` creados (mismo código que la parte d).

Notas:

- Puede implementar operaciones auxiliares en las clases dadas en el diagrama si considera que facilitan la resolución de las operaciones pedidas.
- Se puede utilizar el tipo `std::string` [6] para implementar los atributos de tipo string, así como estructuras de datos de la biblioteca STL [7], tales como vector, set, map, etc.
- Se deben solucionar los problemas de dependencias circulares entre las clases, por ejemplo con relaciones bidireccionales. Para ello es necesario utilizar declaraciones en avanzada (forward declarations) [8].

Referencias

[1] Programación 4. Instructivo de Compilación
URL: https://eva.fing.edu.uy/course/view.php?id=413&section=3

[2] EVA Programación 4.
URL: https://eva.fing.edu.uy/course/view.php?id=413

[3] C++98 Standard en GCC
URL:https://gcc.gnu.org/projects/cxx-status.html#cxx98

[4] C++.
URL: https://www.cplusplus.com/

[5] Unidad de Recursos Informáticos, Salas Linux.
URL: https://www.fing.edu.uy/es/sysadmin/ensenanza/salas-linux

[6] Tipo `std::string`
URL: https://en.cppreference.com/w/cpp/string/basic_string

[7] C++ Standard Template Library (STL)
URL: https://cplusplus.com/reference/stl/

[8] Programación 4. Referencias Circulares y Namespaces
URL: https://eva.fing.edu.uy/course/view.php?id=413&section=3